

第 2 章 运算符重载

所谓重载，就是重新赋予新的含义。重载机制在各种语言中都普遍使用。如目前网络语言就是采用“重载”机制，对某些词汇的传统意义赋予了“新的含义”。例如，“杯具”一词，其传统意义是喝茶的器具，但现在在网络语言中将其重载为“悲伤”或“悲剧”等含义，从而实现了“一词多义”。在 C++ 语言中，对运算符、函数名等标识也可以引入重载机制。例如，函数重载就是对一个已有函数赋予新的含义，使之实现新功能。前一章介绍的构造函数重载就是函数重载的典型实例。本章将重点介绍 C++ 语言中的运算符重载。

2.1 运算符重载的一般方法

前面章节中已经学习过，C++ 所有运算符的操作数据都有类型限定，且对不同类型的数据分别有不同的操作功能呢。目前，编译系统主要是对基本类型数据（如 `int`, `float`, `char` 等）进行操作功能的定义。但对用户自定义类型数据（如结构体变量）就无法直接使用这些运算符来进行运算。例如，无法采用“+”运算符对 2 个结构体变量直接相加。因此，如何使得 C++ 的这些运算符也能对用户自定义类型数据或对象直接运算呢？这就要使用 C++ 的运算符重载机制。下面就通过一个实际例子来进一步阐述什么是运算符重载。

实例 2-1 申明一个二维向量类（`Vector`），并实现对 2 个二维向量对象实现坐标相加运算。

```
#include <iostream>
using namespace std;

class Vector //申明向量类Vector
{
public:
    Vector() {x=0;y=0;} //定义向量类的无参构造函数
    Vector(double xx, double yy) {x=xx;y=yy;} //定义有参构造函数
    Vector vector_add(Vector &v2); //声明向量对象相加函数
    void display();
private:
    double x;
    double y;
};

Vector Vector::vector_add(Vector &v2) //向量相加函数的定义
{
    Vector v;
    v.x=x+v2.x;
    v.y=y+v2.y;
    return v;
}

void Vector::display() //向量输出函数的定义
```

```
{cout<<"("<<x<<","<<y<<")"<<endl;}
```

```
int main()
{
    Vector vec1(3,4),vec2(5,-10),vec3;
    vec3=vec1.vector_add(vec2); //调用向量相加函数实现个对象相加
    cout<<"vec1="; vec1.display(); //输出vec1的值
    cout<<"vec2="; vec2.display(); //输出vec2的值
    cout<<"vec1+vec2="; vec3.display(); //输出vec3的值
    return 0;
}
```

运行结果如下:

```
vec1=(3,4)
vec2=(5,10)
vec1+vec2=(8,-6)
```

在上述程序中，vector_add() 函数的函数体中有 2 条语句“v.x=x+v2.x;”和“v.y=y+v2.y;”分别是采用“+”运算符来直接对两个坐标(double)数据相加。这是 C++ 编译系统规定的“+”运算符基本功能。但是是否可以采用“vec3=vec1+vec2;”来代替“vec3=vec1.vector_add(vec2);”来直接实现 2 个向量对象的相加呢？为此，C++ 引入了运算符重载机制。

所谓运算符重载是对已有的运算符赋予多重含义，使同一个运算符作用于不同类型的数据导致不同类型的行为。运算符重载的实质是函数重载。在实现过程中，首先把指定的运算表达式转化为对运算符函数的调用，运算对象转化为运算符函数的实参，然后根据实参的类型来确定需要调用达函数，这个过程在编译过程中完成。

可能有同学还会问：既然将 2 个对象相加可以采用一个函数 vector_add() 来完成，为什么还要引入运算符重载机制呢？虽然重载运算符所实现的功能完全可以用函数实现。但通过 vector_add() 函数调用来实现两个数相加很不直观，使用也不方便。使用运算符重载能使用户程序易于编写、阅读。而且，在实际工作中，类的声明和类的使用往往是分离的。假如在声明 Vector 类时，对运算符+，-，*都进行了重载，那么使用这个类的用户在编程时可以完全不考虑函数是怎么实现的，放心大胆地直接使用+，-，*进行复数的运算即可，十分方便。

重载运算符的函数一般格式如下：

函数类型 operator 运算符名称(形参列表)

{ 对运算符的重载操作语句 }

例如，对于上述程序中，如果需要将“+”用于 Vector 类（向量）对象的加法运算，该运算符重载的申明语句如下：

```
Vector operator+ (Vector & c1, Vector & c2);
```

在该语句中，可以将“operator+ (Vector & c1, Vector & c2)”看做为一个函数名，可以说：采用了 operator+() 函数来重载运算符“+”。

下面采用运算符重载机制重新改写程序实例 2-1 代码如下。

实例 2-2 基于运算符重载机制的的向量类申明及其对象相加操作。

```
#include <iostream>
using namespace std;
```

```

class Vector
{
public:
    Vector() {x=0;y=0;}
    Vector(double xx, double yy) {x=xx;y=yy;}
    Vector operator+(Vector &v2); //声明重载运算符“+”的函数
    void display();
private:
    double x;
    double y;
};

Vector Vector::operator+(Vector &v2) //定义重载运算符的函数
{
    Vector v;
    v.x=x+v2.x;
    v.y=y+v2.y;
    return v;
}

void Vector::display()
{cout<<"("<<x<<" "<<y<<"")<<endl;}

int main()
{
    Vector vec1(3,4), vec2(5,-10), vec3;
    vec3=vec1+vec2; //运算符+用于向量运算
    cout<<"vec1="; vec1.display();
    cout<<"vec2="; vec2.display();
    cout<<"vec1+vec2="; vec3.display();
    return 0;
}

```

运行结果与实例 2-1 完全相同：

```

vec1=(3,4)
vec2=(5,10)
vec1+vec2=(8,-6)

```

通过分析上述实例 2-1 和 2-2，两者差异如下：

(1) 在实例 2-2 中，使用 operator+() 函数取代了实例 2-1 中的 vector_add() 函数。而且，只是函数名不同，函数体和函数返回值的类型都是相同的。

(2) 在 main 函数中，以 “vec3=vec1+vec2;” 取代了实例 2.1 中的 “vec3=vec1.vector_add(vec2);”。在将运算符+重载为类的成员函数后，C++编译系统将

程序中的表达式“vec1+vec2”解释为 vec1.vector_add(vec2)，即以对象 vec2 为实参调用对象 vec1 的运算符重载函数 operator+(Vector &v2)，进行求值，得到两个对象之和。

下面对 C++运算符重载规则总结如下 4 项规定。

(1) 运算符被重载后，其原有的功能仍然保留，没有丧失或改变。通过运算符重载，只是扩大了 C++已有运算符的作用范围，使之能用于类对象或自定义数据类型。运算符重载是针对新类型数据的实际需要，对原有运算符进行适当的改造。一般来说，重载的功能应当与原有功能相类似，不能改变原运算符的操作对象个数，同时至少要有一个操作对象是自定义类型。

(2) C++不允许用户自己定义新的运算符，只能对已有的 C++运算符进行重载。重载之后运算符的优先级和结合性都不会改变。

(3) C++中绝大部分的运算符允许重载。具体情况见表 2-1，但也有 5 个运算符不能重载。分别是：.（成员访问运算符）；*（成员指针访问运算符）；::（域运算符）；sizeof（长度运算符）；?:（条件运算符）。

表 2.1 C++允许重载的运算符

双目算术运算符	+（加），-（减），*（乘），/（除），%（取模）
关系运算符	==（等于），!=（不等于），<（小于），>（大于），<=（小于等于），>=（大于等于）
逻辑运算符	（逻辑或），&&（逻辑与），！（逻辑非）
单目运算符	+（正），-（负），*（指针），&（取地址）
自增自减运算符	++（自增），--（自减）
位运算符	（按位或），&（按位与），~（按位取反），^（按位异或），<<（左移），>>（右移）
赋值运算符	=，+=，-=，*=，/=，%=，&=， =，^=，<<=，>>=
空间申请与释放	new，delete，new[]，delete[]
其他运算符	（）（函数调用），->（成员访问），->*（成员指针访问），，（逗号），[]（下标）

(4) 用于类对象的运算符一般必须重载。但运算符“=”和“&”不必用户重载，编译器内置了自动重载机制。赋值运算符(=)可以用于任何类的同类对象之间相互赋值。地址运算符&可以返回任何类对象在内存中的地址。

C++运算符的载对机制很好地把运算符重载和类结合起来，使 C++具有更强大的功能、更好的可扩充性和适应性，这是 C++最吸引人的特点之一。

2.2 运算符重载函数作为类的成员函数或友元函数

在前节的实例 2-2 中对运算符“+”进行了重载，使之能用于两个向量（对象）的相加。在该例中运算符重载函数 operator+作为 Vector 类中的成员函数。那么“+”是双目运算符，为什么其重载函数中只有一个参数呢？

事实上，有一个参数是隐含的。由于重载函数是 Vector 类中的成员函数，运算符函数是用 this 指针隐式地访问类对象的成员。另一个是形参对象中的成员。如 this->x+vec2.x。其中 this->x 就是 vec1.x。

因此，如果将运算符函数重载为类的成员函数后，会隐含一个操作数，即本类的对象。这样有时不是很直观。是否可以使得运算符重载函数显式地指明两个操作数呢？答案是肯定的。因为 C++ 规定，运算符重载函数可以是类的成员函数，也可以是类的友元函数。

下面就将实例 2-2 中 “+” 运算符重载函数改写为 Vector 类的友元函数来实现。注意对比两者差异。

实例 2-3 采用友元函数来实现 “+” 运算符重载。

```
#include <iostream>
using namespace std;

class Vector
{
public:
    Vector() {x=0;y=0;}
    Vector(double xx, double yy) {x=xx;y=yy;}
    friend Vector operator + (Vector &v1, Vector &v2); //作为友元函数
    void display();
private:
    double x;
    double y;
};

Vector operator + (Vector &v1, Vector &v2) //友元函数的定义
{
    return Vector(v1.x+v2.x, v1.y+v2.y);
}

void Vector::display()
{cout<<"("<<x<<","<<y<<")"<<endl;}

int main()
{
    Vector vec1(3, 4), vec2(5, -10), vec3;
    vec3=vec1+vec2;
    cout<<"vec1="; vec1.display();
    cout<<"vec2="; vec2.display();
    cout<<"vec1+vec2="; vec3.display();
    return 0;
}
```

对比分析可以发现：本程序中只作了一处改动：将运算符重载函数不作为类的成员函数，而是把它放在类外作为普通函数来申明。再在 `Vector` 类中声明它为友元函数。这时，该运算符函数就必须有两个参数。在这样的情况下，C++编译系统将程序中的表达式“`vec1+vec2`”解释为 `operator+(vec1,vec2)`，即执行以下函数：

```
Vector operator + (Vector &v1, Vector &v2)
{return Vector(v1.x+v2.x, v1.y+v2.y);}
```

提问：运算符重载函数是否可以是非类的成员函数也不是友元函数的普通函数？例如，在程序实例 2-3 中，如果不将 `Vector operator + (Vector &v1, Vector &v2)` 申明为类的友元，是否可以？

原则上是可以的。但一般情况下，极少使用既不是类的成员函数也不是友元函数的普通函数。原因是上面提到的，普通函数不能直接访问类的私有成员。

那么，运算符重载如何选择成员函数还是友元函数呢？如果将运算符重载函数作为成员函数，它可以通过 `this` 指针自由地访问本类的数据成员。因此，可以少写一个函数的参数。但必须要求运算表达式第一个参数（即运算符左侧的操作数）是一个类对象。而且，与运算符函数的类型相同。因为必须通过类的对象去调用该类的成员函数。另外，只有运算符重载函数返回值与该对象同类型，运算结果才有意义。有关这个问题在后面介绍类型转换构造函数时还将深入讨论。

如果实际需要在重载运算符时，运算符左侧的操作数是其他类型（如表达式 `i*v2`，`i` 是整数），这时运算符重载函数不能作为成员函数，只能作为非成员函数。如果函数需要访问类的私有成员，则必须声明为友元函数。可以在 `Vector` 类中声明：`friend Vector operator*(int &i,Vector &v)`。

因此，将双目运算符重载为友元函数时，在函数的形参表列中必须有两个参数。但形参的顺序任意，不要求第一个参数必须为类对象。但在使用运算符的表达式中，要求运算符左侧的操作数与函数第一个参数对应，运算符右侧的操作数与函数的第二个参数对应。此时，数学上的交换律在此不适用。如果希望适用交换律，则应再重载一次运算符“+”。如 `Vector operator*(Vector &v, int &i)`。这样使用表达式 `i*v2` 和 `v2*i` 都合法，编译系统会根据表达式的形式选择调用与之匹配的运算符重载函数。当然，也可以采用后面介绍的类型转换重载函数等机制来满足交换律。

另外，C++有些运算符就只能为类成员函数，或者友元等。例如，C++规定：赋值运算符、下标运算符、函数调用运算符等必须定义为类的成员函数。而流插入“<<”和流提取运算符“>>”、类型转换运算符等则不能定义为类的成员函数。其中原因，在相关章节将会介绍。

2.2.1 双目运算符的重载应用举例

由于友元的使用会破坏类的封装。因此，从原则上说，要尽量将运算符函数作为成员函数。但考虑到各方面的因素，一般将单目运算符重载为成员函数，将双目运算符重载为友元

函数。下面结合一个字符串类 String 来介绍双目运算符的重载应用方法。

实例 2-4 继续为前面出现过的 Vector 类添加更多的运算符。我们不妨规定，向量的大小，可以通过模的长短来进行比较，从而对“==”“<”和“>”运算符的含义进行约定。下面，将通过运算符重载机制，来实现两个向量的等于、小于和大于的比较运算。

```
#include <iostream>
using namespace std;

class Vector
{
public:
    Vector() {x=0;y=0;}
    Vector(double xx, double yy) {x=xx;y=yy;}
    friend bool operator>(Vector &vec1, Vector &vec2); //运算符>重载
    friend bool operator<(Vector &vec1, Vector &vec2); //运算符<重载
    friend bool operator==(Vector &vec1, Vector &vec2); //运算符==重载
    void display();
private:
    double x;
    double y;
};

bool operator>(Vector &vec1, Vector &vec2)
{
    double r1=vec1.x*vec1.x+vec1.y*vec1.y;
    double r2=vec2.x*vec2.x+vec2.y*vec2.y;
    if(r1>r2)
        return true;
    else
        return false;
}

bool operator<(Vector &vec1, Vector &vec2)
{
    double r1=vec1.x*vec1.x+vec1.y*vec1.y;
    double r2=vec2.x*vec2.x+vec2.y*vec2.y;
    if(r1<r2)
        return true;
    else
        return false;
}

bool operator==(Vector &vec1, Vector &vec2)
{

```

```

double r1=vec1.x*vec1.x+vec1.y*vec1.y;
double r2=vec2.x*vec2.x+vec2.y*vec2.y;
if(r1==r2)
    return true;
else
    return false;
}

void Vector::display()
{cout<<"("<<x<<"", "<<y<<"")";}

void compare(Vector &vec1, Vector &vec2)
{
    if(operator>(vec1, vec2)==1)
        {vec1.display();cout<<">";vec2.display();}
    else if(operator<(vec1, vec2)==1)
        {vec1.display();cout<<"<";vec2.display();}
    else if(operator==(vec1, vec2)==1)
        {vec1.display();cout<<"=";vec2.display();}
    cout<<endl;
}

int main()
{
    Vector vec1(1, 2), vec2(1, 1), vec3(2, 3);
    compare(vec1, vec2);
    compare(vec2, vec3);
    compare(vec1, vec1);
    return 0;
}

```

运行结果为

(1, 2)>(1, 1)

(1, 1)<(2, 3)

(1, 2)=(1, 2)

通过上述程序分析，可以感受到 C++ 的运算符重载功能的方便和好处。

2.2.2 单目运算符的重载举例

单目运算符只有一个操作数，例如：!a、-b、&c、*p 等。还有最常用的++i 和--i。重载单目运算符的方法与重载双目运算符的方法是类似的。但由于单目运算符只有一个操作数，因此，运算符重载函数只有一个参数。如果运算符重载函数作为成员函数，则还可省略此参数。下面以自增运算符“++”和下标运算符“[]”等为例，介绍单目运算符的重载。

1. 自增运算符的重载

实例 2-5 申明一个 Clock 类，包含数据成员 minute（分）和 sec（秒），模拟秒表。每次走一秒，满 60 秒进一分钟。此时秒又从 0 开始算。并采用++或--对 Clock 类对象求进行自增或自减运算。

```
#include <iostream>
#include <iomanip>
#include <Windows.h>
using namespace std;

class Clock
{
public:
    Clock ( ) {minute=0;sec=0;} //默认构造函数
    Clock (int m,int s):minute(m),sec(s){ } //构造函数重载
    Clock operator++( ) ; //声明运算符重载函数
    void display( ) {cout<<setw(2)<<setfill('0')<<minute<<":"
                    <<setw(2)<<setfill('0')<<sec<<endl;} //定义输出时间函数
private:
    int minute;
    int sec;
};

Clock Clock::operator++( ) //定义运算符重载函数
{
    if(++sec>=60)
    {
        sec-=60; //满秒进分钟
        ++minute;
    }
    return *this; //返回当前对象值
}

int main( )
{
    Clock time1(34,0);
    for (int i=0;i<61;i++)
    {
        ++time1;
        time1.display( );
        Sleep(1000);
        system("cls");
    }
    return 0;
}
```

```
}
```

运行情况如下：

```
34:01
34:02
⋮
34:59
35:00
35:01（共输出了 61 行）
```

可以看到：在程序中对运算符“++”进行了重载，使它能用于 Clock 类对象。然而，上述程序中，使用的是“++”的前置形式。前面章节中学习过：“++”和“--”运算符有两种使用方式，前置自增运算符和后置自增运算符。它们的作用是不一样的，在重载时怎样区别这二者呢？针对该特点，C++约定：在自增（自减）运算符重载函数中，增加一个 int 型形参，就是后置自增（自减）运算符函数。下面通过实例 2-6 来介绍“++”的后置形式的重载方法。

实例 2-6 自增运算符“++”的后置形式的重载应用。

```
#include <iostream>
using namespace std;
class Clock
{
public:
    Clock ( ) {minute=0;sec=0;}
    Clock (int m, int s):minute(m), sec(s) {}
    Clock operator++( ) ;//声明前置自增运算符“++”重载函数
    Clock operator++(int) ;//声明后置自增运算符“++”重载函数
    void display( ) {cout<<minute<<":"<<sec<<endl;}
private:
    int minute;
    int sec;
};

Clock Clock::operator++( )    //定义前置自增运算符“++”重载函数
{
    if(++sec>=60)
    {
        sec-=60;
        ++minute;
    }
    return *this;    //返回自加后的当前对象
}

Clock Clock::operator++(int)    //定义后置自增运算符“++”重载函数
{
```

```

    Clock temp(*this);
    sec++;
    if(sec>=60)
    {
        sec-=60;
        ++minute;
    }
    return temp; //返回的是自加前的对象
}

int main( )
{
    Clock time1(34,59),time2;
    cout<<" time1 : ";
    time1.display( );
    ++time1;
    cout<<"++time1: ";
    time1.display( );
    time2=time1++; //将自加前的对象的值赋给time2
    cout<<"time1++: ";
    time1.display( );
    cout<<" time2 :";
    time2.display( ); //输出time2对象的值
    return 0;
}

```

运行结果如下：

```

time1 : 34:59 (time1 原值)
++time1: 35:0 (执行++time1 后 time1 的值)
time1++: 35:1 (再执行 time1++后 time1 的值)
time2 : 35:0 (time2 保存的是执行 time1++前 time1 的值)

```

可以看到：重载后置自增运算符时，多了一个 int 型的参数，增加这个参数只是为了与前置自增运算符重载函数有所区别，此外没有任何作用。编译系统在遇到重载后置自增运算符时，会自动调用此函数。另外，对比分析程序实例 2-5 和 2-6，注意前置自增运算符“++”和后置自增运算符“++”的区别。前者是先自加，返回的是修改后的对象本身（return *this;）。后者返回的是自加前的临时对象（return temp），然后对象自加。

2. 下标运算符[]的重载

下标运算符是 C 和 C++语言常用的数据运算符。但是，在 C 语言中，下标运算符[]没有定义下标越界的检测功能。而这种下标越界的错误很容易产生。因此，通过运算符重载，对下标运算符增加一个越界检测功能。具体程序如下实例 2-7 所示。

```

#include <iostream>

```

```

#include <assert.h>
using namespace std;

class Matrix
{
    double *data;
    int row;
    int col;
public:
    Matrix();
    Matrix(double * pData, int r, int c);
    ~Matrix();
    double & operator() (int r, int c);
};

Matrix::Matrix()
{
    data = NULL;
    row = 0;
    col = 0;
}

Matrix::Matrix(double * pData, int r, int c)
{
    data = pData;
    row = r;
    col = c;
}

Matrix::~Matrix()
{
    delete []data;
}

double & Matrix::operator() (int r, int c)
{
    assert(r >= 0 && r < row && c >= 0 && c < col);
    return data[col * r + c];
}

int main()
{
    double * pData = new double[9];
    int row = 3;

```

```

    int col = 3;
    Matrix m(pData, row, col);

    int cnt = 1;
    for(int i = 0; i < row; i++)
    {
        for(int j = 0 ; j < col; j++)
        {
            m(i, j) = cnt++;
            cout<<m(i, j)<<' ';
        }
        cout<<endl;
    }

    return 0;
}

```

3. 函数调用运算符()的重载

下面通过对函数调用运算符()的重载，使得能实现矩阵类的多重下标访问。程序如下实例 2-8 所示。

```

#include <iostream>
#include <assert.h>
using namespace std;

class Vector
{
    int * data;
    int size;
public:
    Vector();
    Vector(int * pData, int n);
    ~Vector();
    int SetSize(int n);
    int GetSize() const;
    int & operator [] (int n);
};

Vector::Vector()
{
    data = NULL;
    size = 0;
}

Vector::Vector(int * pData, int n)
{

```

```

        data = pData;
        size = n;
    }

Vector::~Vector()
{
    delete []data;
}

int Vector::GetSize() const
{
    return size;
}

int Vector::SetSize(int n)
{
    delete []data;
    data = new int[n];
    size = n;
    return size;
}

int & Vector::operator[](int n)
{
    assert(n >= 0 && n < size);
    return data[n];
}

int main()
{
    Vector v;
    int size = 10;
    v.SetSize(size);

    for(int i = 0; i < size; i++)
        v[i] = i;

    cout<<v[3]<<endl;
    cout<<v[10]<<endl;

    return 0;
}

```

2.2.3 流插入运算符和流提取运算符的重载

在本教材最后章节中将会详细地介绍：所有 C++编译系统都在类库中提供输入流类 `istream` 和输出流类 `ostream`。`cin` 和 `cout` 分别是 `istream` 类和 `ostream` 类的对象。在类库提供的头文件中已经对 “<<” 和 “>>” 这 2 个运算符进行了重载，使之作为流插入运算符和流提取运算符，能用来输出和输入 C++标准类型的数据。这就是为什么在本书前面几章中，凡是用 “`cout<<`” 和 “`cin>>`” 对标准类型数据进行输入输出时，都必须使用 `#include <iostream>` 头文件包含语句。

但是用户自己定义的类型的数据，是不能直接用 “<<” 和 “>>” 来输出和输入的。也就是说，对用户自定义类型的数据（包括类对象），如果想用它们输出和输入，用户必须对它们重载。而且只能将重载 “>>” 和 “<<” 的函数作为友元函数，而不能将它们定义为成员函数。因此，“<<” 和 “>>” 重载的函数形式只能如下（至于为什么非得使用上述格式的原因，会在最后流 I/O 操作的相关章节做深入介绍）。

```
istream & operator >> (istream &, 自定义类 &);
```

```
ostream & operator << (ostream &, 自定义类 &);
```

其中：重载运算符 “>>” 的函数的第一个参数和函数的类型都必须是 `istream&` 类型，第二个参数是要进行输入操作的类。重载 “<<” 函数的第一个参数和函数的类型都必须是 `ostream&` 类型，第二个参数是要进行输出操作的类。

下面通过前面的向量操作实例来介绍这 2 个运算符的重载方法。

实例 2-9 在实例 2-2 的基础上，用重载 “<<” 和 “>>” 运算符进行向量的输入和输出。

```
#include <iostream>
using namespace std;

class Vector
{
public:
    Vector() {x=0;y=0;}
    Vector(double xx, double yy) {x=xx;y=yy;}
    Vector operator+(Vector &v2); //运算符“+”重载为成员函数
    friend ostream& operator << (ostream&, Vector&); //重载运算符“<<”
    friend istream& operator >> (istream&, Vector&); //重载运算符“>>”
private:
    double x;
    double y;
};

Vector Vector::operator+(Vector &v2) //定义重载运算符的函数
{
    Vector v;
```

```

        v.x=v.x+v2.x;
        v.y=v.y+v2.y;
        return v;
    }

ostream& operator << (ostream& output, Vector & v) //定义重载运算符 "<<"
{
    output<<"("<<v.x<<","<<v.y<<")";
    return output;
}

istream& operator >> (istream& input, Vector & v) //定义重载运算符 ">>"
{
    cout<<"input x and y of vector:";
    input>>v.x>>v.y;
    return input;
}

int main()
{
    Vector vec1,vec2,vec3;
    cin>>vec1>>vec2;
    vec3=vec1+vec2;
    cout<<"vec1="<<vec1<<endl;
    cout<<"vec2="<<vec2<<endl;
    cout<<"vec3="<<vec3<<endl;
    return 0;
}

```

运行情况如下：

```

input x and y of vector:5 5
input x and y of vector:6 6
vec1=(5,5)
vec2=(6,6)
vec3=(11,11)

```

通过分析上述程序，可以发现：在对运算符“<<”和“>>”重载后，在程序中用“<<”不仅能进行标准类型数据输入和输出。而且，可以进行用户自己定义数据（类对象）的输入和输出。与程序实例 2-2 中使用 display() 函数来进行复数对象（如 vec3）的输出相比，形式直观，可读性好，易于使用。

另外，通过对运算符“<<”和“>>”重载应用实例分析，我们还需要体会到，在运算符重载中使用引用(reference)的重要性。采用引用作为函数的形参可以在调用函数的参数传递过程中不是传递值，而是通过传址方式使形参成为实参的别名。此时，形参

别名与实参代表同一内存空间。因此不生成临时变量（实参的副本），减少了程序执行的时间和空间的开销。

如果重载函数的返回值也是使用对象的引用，返回的是引用所代表的对象，可以被赋值或参与其他操作（如保留 cout 流的当前值以便能连续使用“<<”输出）。

一般来说，函数的返回值大体可以分为3种：返回非引用，返回引用，返回引用左值。具体介绍如下：

1) 返回非引用类型：函数的返回值用于初始化在跳出函数时候创建的临时对象。用函数返回值来初始化临时对象与用实参初始化形参的方法是一样的。如果返回类型不是引用的话，在函数返回的地方，会将返回值复制给临时对象。且其返回值既可以是局部对象，也可以是表达式的结果。例如：

```
string make_plural(size_t i,const string &word,const string &ending)
{ return (i==1)?word:word+ending; }
```

以上函数，当i等于1时，函数返回word形参的副本；当i不等于1时函数返回一个临时的string对象，这个临时对象是由字符串word和ending相加而成的。这两种情况下，return都在调用该函数的地方复制了返回的string对象。

2) 返回引用：当函数返回引用类型的时候，没有复制返回值，而是返回对象的引用（即对象本身）。函数返回引用时，可以利用全局变量（作为函数返回），或者在函数的形参表中有引用或者指针（作为函数返回），这两者有一个共同点，就是返回执行完毕以后，变量依然存在，那么返回的引用才有意义。因此，不允许返回局部变量的引用。另外，当不希望返回的对象被修改的时候，可以添加const。例如：

```
const string &shorterString(const string &s1,const string &s2)
{
    return s1.size()<s2.size()?s1:s2;
}
```

以上函数的返回值是引用类型。无论返回s1或是s2,调用函数和返回结果时，都没有复制这些string对象。

注意：不要返回局部对象的引用

```
const string &mainip(const string &s)
{
    string ret=s;
    return ret;
}
```

当函数执行完毕，程序将释放分配给局部对象的存储空间。此时，对局部对象的引用就会指向不确定的内存。同理，也不能返回局部对象的指针。

3) 返回引用左值。函数返回引实际上是一个变量的内存地址。既然是内存地址的话，那么肯定可以读写该地址所对应的内存区域的值，即就是“左值”，可以出现在赋值语句的左边。

```
char &get_val(string &str,string::size_type ix)
{
    return str[ix];
}
```

使用语句调用：

```
string s("123456");
```

```
cout<<s<<endl;
get_val(s,0)='a';
cout<<s<<endl;
把函数应用于左值。
```

2.3 类对象与标准类型数据之间的转换方法

2.3.1 转换构造函数

前面已经学习过：在 C++ 中，某些不同类型数据之间可以自动转换，例如：`int i = 6; i = 7.5 + i;` 编译系统对 7.5 是作为 double 型数处理的。在求解表达式时，先将 6 转换成 double 型，然后与 7.5 相加，得到和为 13.5，在向整型变量 i 赋值时，将 13.5 转换为整数 13，然后赋给 i。这种转换是由 C++ 编译系统自动完成的，用户不需干预。这种转换也称为隐式类型转换。

另外，C++ 还提供强制类型转换，程序人员在程序中指定将一种指定的数据转换成另一指定的类型，如 `int(89.5)`，编译系统将 89.5 转换为整型数 89。

那么，如果需要某标准类型数据转换为某类的对象，编译系统能进行处理吗？解决问题的关键是让编译系统知道怎样去进行这些转换，需要定义专门的函数来处理，即所谓的“转换构造函数 (conversion constructor function)”。

转换构造函数也是类的构造函数一种，其作用是将一个其他类型的数据转换成一个类的对象。例如，如果需要将一个标准 int 数据转为复数 Complex 类的对象，则需要为 Complex 类定义一个类型转换构造函数。例如：`Complex(double r) {real=r;imag=0;}`。

其作用是将 double 型的参数 r 转换成 Complex 类的对象，将 r 作为复数的实部，虚部为 0。用户可以根据需要定义转换构造函数，在函数体中告诉编译系统怎样去进行转换。

转换构造函数的特点可以归纳如下：

- (1) 转换构造函数是类的构造函数一种，符合构造函数的所有特征。
- (2) 转换构造函数只有一个形参，它是需要被转换的数据类型。在函数体中指定转换的方法。
- (3) 在该类的作用域内可以用以下格式进行类型转换：类名(指定类型的数据)。如 `Complex_Data data; data=Complex_Dat (3.7)`。这样就可以将指定类型（如 double）的数据 3.7 转换为 Complex_Dat 类的一个对象。

当然，转换构造函数不仅可以将一个标准类型数据转换成类对象，也可以将另一个类的对象转换成转换构造函数所在的类对象。例如，假设有一个学生类 Student 的对象 s，如果需要将 s 转换为教师类 Teacher 的对象，可以在 Teacher 类中申明如下的转换构造函数：

```
Teacher(Student& s) {num=s.num;strcpy(name,s.name);sex=s.sex;}
```

但应注意：Student 类对象 s 中的 num, name, sex 必须是公用成员，否则不能被 Teacher

类中声明的转换构造函数引用。

提问：类型转换构造与复制构造函数的区别与联系？

2.3.2 基于运算符重载机制的类型转换函数

用转换构造函数可以将一个指定类型的数据转换为类的对象。如果需要反过来将一个类的对象转换为一个标准类型数据（例如将一个 `Complex_Data` 类对象转换成 `double` 类型数据），又该如何办呢？

回顾一下前面介绍的标准类型的强制转换方式，例如，`double(3)`；C++编译系统能将整型（当然也可以其他标准类型）数据转换为 `double`。如果将标准类型符（例如 `double()`）也理解为运算符（姑且称为类型运算符）的话，是否可以采用运算符重载机制来扩展这些类型运算符的强制类型转换功能，使得不仅能在标准类型之间进行转换，而且还可以将用户自定义的类对象数据也可转换为某一标准类型呢？

答案是肯定的。C++提供类型转换函数(type conversion function)就基于运算符重载机制，将一个类的对象转换成另一标准类型的数据。例如，假设在前面介绍的 `Complex_Data` 类中，可以定义一个类型转换函数：

```
operator double()  
{return real;}
```

该函数 `double()` 就可以将 `Complex_Data` 类对象转换为 `double` 类型数据。`double` 类型转换经过重载后，除了原有的含义外，还获得新的含义（将一个 `Complex_Data` 类对象转换为 `double` 类型数据，并指定了转换方法）。

可以归纳出类型转换函数定义的一般格式为：

```
operator 类型名()  
{实现转换的语句}
```

需要特别注意：

（1）在函数名前面不能指定函数类型，函数没有参数。其返回值的类型是由函数名中指定的类型名来确定的。

（2）类型转换函数只能作为成员函数，因为转换的主体是本类的对象。不能作为友元函数或普通函数。

（3）转换构造函数和类型转换运算符有一个共同的功能：当需要的时候，编译系统会自动调用这些函数，建立一个无名的临时对象（或临时变量）。

实例 2-10 使用类型转换函数的程序举例。

```
#include <iostream>  
using namespace std;
```

```

class Vector
{
public:
    Vector() {x=0;y=0;}
    Vector(double xx, double yy) {x=xx;y=yy;}
    friend Vector operator+ (Vector v1, Vector v2); //定义运算符 “+” 重载函数
    operator double() {return sqrt(x*x+y*y);} //类型转换函数
private:
    double x;
    double y;
};

Vector operator+(Vector v1, Vector v2) //定义重载运算符的函数
{
    Vector v;
    v.x=v1.x+v2.x;
    v.y=v1.y+v2.y;
    return v;
}

int main()
{
    Vector vec1(3,4), vec2(5,-10);
    double d1, d2;
    d1=2.5+vec1; //要求将一个double数据与Vector类数据相加
    d2=vec1+vec2; //要求将两个Vector类对象相加，赋值给d
    cout<<d1<<endl;
    cout<<d2<<endl;
    return 0;
}

```

分析:

(1) 如果在 Vector 类中没有定义类型转换函数 operator double, 程序编译将在 d1=2.5+vec1 和 d2=vec1+vec2 语句出错。为什么?

(2) 语句 d1=2.5+vec1 执行时, 编译系统自动调用类型转换函数 double, 先将 vec1 转为 double 类型数据, 然后与 2.5 相加, 结果为 double 且赋值给 double 型变量 d。

(3) 语句 d2=vec1+vec2 执行时, 编译系统将 vec1 与 vec2 两个类对象相加, 得到一个临时的 Vector 类对象, 由于它不能赋值给 double 型变量, 而又有对 double 的重载函数, 于是调用此函数, 把临时类对象转换为 double 数据, 然后赋给 d。

使用类型转换函数的好处: 假如程序中需要对一个 Vector 类对象和一个 double 型变量进行+, -, *, /等算术运算, 以及关系运算和逻辑运算, 如果不用类型转换函数, 就要对多种运算符进行重载, 以便能进行各种运算。这样是十分麻烦的, 工作量较大, 程序显得冗长。

如果用类型转换函数对 double 进行重载（使 Vector 类对象转换为 double 型数据），就不必对各种运算符进行重载，因为 Vector 类对象可以被自动地转换为 double 型数据，而标准类型的数据的运算，是可以使用系统提供的各种运算符的。

2.3.3 综合程序举例

实例 2-11 中同时引入了转换构造函数、运算符重载函数和类型转换函数，分析这些函数在程序的作用。

```
#include <iostream>
using namespace std;

class Vector
{
public:
    Vector() {x=0;y=0;} //默认构造函数
    Vector(double xx, double yy) {x=xx;y=yy;} //实现初始化的构造函数
    Vector (double r) {x=r;y=0;} //转换构造函数
    friend Vector operator+ (Vector v1, Vector v2);
    Vector & operator = (Vector &v1);
    void display();
private:
    double x;
    double y;
};

Vector operator+(Vector v1, Vector v2)
{
    return Vector(v1.x+v2.x, v1.y+v2.y);
}

Vector & Vector::operator = (Vector &v1) {
    return v1;
}

void Vector::display()
{
    cout<<"("<<x<<"", "<<y<<"")"<<endl;
}

int main()
{
    Vector vec1(3, 4), vec2(5, -10), vec3;
    vec3=vec1+5.5; //向量与double数据相加
```

```
    vec3.display( );  
    return 0;  
}
```

程序的分析:

(1) 如果没有定义转换构造函数, 则此程序编译出错。现在, 在类 `Vector` 中定义了转换构造函数。由于已重载了算符 “+”, 在处理表达式 `vec1+5.5` 时, 编译系统把它解释为 `operator+(vec1, 5.5)`。由于 `5.5` 不是 `Vector` 类对象, 系统先调用转换构造函数 `Vector(5.5)`, 建立一个临时的 `Vector` 类对象。上面的函数调用相当于 `operator+(vec1, Vector(5.5))`。

(2) 如果把 “`vec3=vec1+5.5`” 改为 `vec3=5.5+vec1`, 程序可以通过编译和正常运行。过程与前相同。

从中得到一个重要结论: 在已定义了相应的转换构造函数情况下, 将运算符 “+” 函数重载为友元函数, 在进行两个复数相加时, 可以用交换律。如果 “+” 运算符函数重载为成员函数, 它的第一个参数必须是本类的对象。当第一个操作数不是类对象时, 不能将运算符函数重载为成员函数。如果将运算符 “+” 函数重载为类的成员函数, 交换律不适用。由于这个原因, 所以前面有一个基本建议: 一般情况下将双目运算符函数重载为友元函数。单目运算符则多重载为成员函数。

(3) 在上面程序的基础上增加类型转换函数: `operator double() {return sqrt(x*x+y*y);}`。其余部分不变。程序在编译时出错, 为什么? 如何修改? (提示: 可以使用显示类型转换规则)。

(4) 如果在 (3) 的基础上, 将 `friend Vector operator + (Vector vec1, Vector vec2);` 语句注释掉。程序是否可以编译通过? `vec3=vec1+5.5` 语句的执行过程如何? (提示: 先调用类型转换, 再调用构造函数)。

实例 2-12. 在实际应用中, 一个系统中的对象数量会经常发生变化。例如, 一个班级中的同学可能转入或转出等。如何管理这些动态变化的数据对象呢? 这就需要使用动态产生对象。本实例通过介绍对多个节点的管理方法来综合展示本章学过的运算符重载一些知识。引入动态节点的管理也是为后续《数据结构》课程承接奠定基础。

下面实例定义一个学生类来描述众多的学生对象。为了对这些众多的学生对象进行管

理，还定义一个学生管理类，在该学生管理类中，定义学生对象所需的主要操作。为了简单起见，本实例规定只能在学生管理类的对象数据最大下标处进行插入和删除操作。实例中类及对象之间关系如图 2-1 所示。

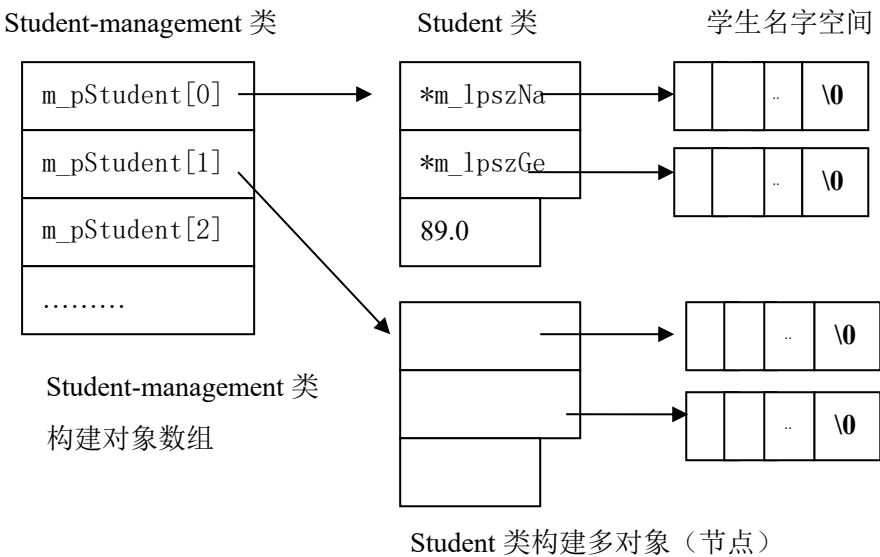


图 2-1 实例中类及对象之间关系

在上述实例中，由于采用动态对象的构建，在释放对象时，需要特别注意的是：先释放学生对象中的学生名字空间和学生性别空间，即delete []m_lpszName，和delete []m_lpszGender。然后，释放学生对象，即delete m_pStudent []。否则就会造成内存溢出。其中的原因，请同学们自己分析。

/******

Filename: student.h

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2015-04-10

Description: Student类的声明，管理一位学生的个人信息

*****/

#ifndef __STUDENT__

#define __STUDENT__

#include <iostream>

#include <assert.h>

using namespace std;

//学生管理类前向声明

class StudentManagement;

//学生类，管理一位学生的个人信息

class Student

```

{
    //友元类声明
    friend StudentManagement;
public:
    //构造函数与析构函数
    Student(char *lpszName, char *lpszGender, int iScore = 0);
    Student(const Student &oStudent);
    virtual ~Student();

    //重载"="符号运算符
    Student &operator=(const Student &oStudent);
public:
    //三个成员分别代表姓名，性别，成绩
    char *m_lpszName;
    char *m_lpszGender;
    int m_iScore;
};

#endif

*****
Filename: studentManagement.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2015-04-10
Description: StudentManagement类的声明，构成一个学生管理栈

*****/
#ifndef __STUDENT_MANAGEMENT__
#define __STUDENT_MANAGEMENT__

#include <iostream>
using namespace std;

//学生类前向声明
class Student;
//常量定义，意义为栈的最大长度
const int P_LENGTH = 20;

//学生管理栈类，以栈的数据结构管理多位学生的信息
class StudentManagement
{
public:
    //构造函数与析构函数

```

```

StudentManagement(int iTotalStudentNum = P_LENGTH);
StudentManagement(const StudentManagement &oStudentManagement);
virtual ~StudentManagement(void);

//插入与删除函数
bool Insert(const Student &oStudent);
bool Del(void);

//判断是否为空的函数
bool IsEmpty(void);

//",", "[", "="运算符的重载函数
const StudentManagement &operator, (const StudentManagement &oStudentManagement);

char *operator[](int i);
StudentManagement &operator=(const StudentManagement &oStudentManagement);

//"new", "delete"运算符的重载函数
void *operator new(size_t size);
void operator delete(void *p);

//"<<", ">>"运算符的重载函数
friend ostream &operator << (ostream &cout, StudentManagement obj);
friend istream &operator >> (istream &cin, StudentManagement &obj);
private:
    Student *m_pStudent[P_LENGTH]; //管理栈
    int m_aLength; //栈长度
    int m_aCurrentTop; //当前栈顶位置
};

#endif

/*****
Filename: main.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2015-04-10
Description: 程序主函数

*****/
#include "Student.h"
#include "Student_Management.h"

```

```

//Summary: 主函数
//
//Parameters:
//      None
//Return: None
//Detail:
void main( )
{
    //使用复合语句形成三个作用域
    {
        //创建学生信息对象
        Student oStudent1( "Tom", "F" );
        Student oStudent2( "Jerry", "F" );

        //创建学生信息管理栈
        StudentManagement oStudentManagement1, oStudentManagement2;

        //向栈中插入
        oStudentManagement1.Insert( oStudent1 );
        oStudentManagement1.Insert( oStudent2 );

        //重载"[]"运算符
        cout << "[" ]运算符重载的结果:" << oStudentManagement1[1] << endl;

        //拷贝构造函数
        StudentManagement oStudentManagement3( oStudentManagement1 );

        //重载",", "=", 运算符
        oStudentManagement2 = oStudentManagement1, oStudentManagement3;

        //重载">>", "<<"运算符
        cin >> oStudentManagement1;
        cout << oStudentManagement1;

        //删除学生信息
        oStudentManagement1.Del( );
        oStudentManagement1.Del( );
    }

    {
        //使用重载后的运算符"new", "delete"进行测试
        StudentManagement *poStudentManagement = new StudentManagement;
        delete poStudentManagement;
    }
}

```

```

    }

    {
        //使用全局运算符"new", "delete"进行测试
        StudentManagement *poStudentManagement = ::new StudentManagement;
        ::delete poStudentManagement;
    }
}

/*****
Filename: student.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2015-04-10
Description: Student类的定义

*****/
#include "Student.h"

//Summary: 构造函数
//
//Parameters:
//    lpszName    姓名
//    lpszGender  性别
//    iScore      成绩, 默认值为
//Return: None
//Detail:
Student::Student(char *lpszName, char *lpszGender, int iScore)
{
    //判断姓名与性别非空
    assert(lpszName);
    assert(lpszGender);

    //采用深拷贝方式复制学生姓名
    m_lpszName = new char[strlen(lpszName)+1];
    if( m_lpszName==NULL )
    {
        cout << "内存分配失败。" << endl;
        exit(0);
    }
    strcpy_s(m_lpszName, strlen(lpszName)+1, lpszName);

    //采用深拷贝方式复制学生性别
    m_lpszGender = new char[strlen(lpszGender)+1];

```

```

        if( m_lpszGender==NULL )
        {
            cout << "内存分配失败。" << endl;
            exit(0);
        }
        strcpy_s(m_lpszGender, strlen(lpszGender)+1, lpszGender);

        m_iScore = iScore;

        cout << "调用了带参构造函数Student: " << m_lpszName << endl;
    }

//Summary: 拷贝构造函数
//
//Parameters:
//    oStudent    待拷贝Student类对象
//Return: None
//Detail:
Student::Student(const Student &oStudent)
{
    //采用深拷贝方式复制学生姓名
    m_lpszName = new char[strlen(oStudent.m_lpszName)+1];
    if( m_lpszName==NULL )
    {
        cout << "内存分配失败。" << endl;
        exit(0);
    }
    strcpy_s(m_lpszName, strlen(oStudent.m_lpszName)+1, oStudent.m_lpszName);

    //采用深拷贝方式复制学生性别
    m_lpszGender = new char[strlen(oStudent.m_lpszGender)+1];
    if( m_lpszGender==NULL )
    {
        cout << "内存分配失败。" << endl;
        exit(0);
    }
    strcpy_s(m_lpszGender, strlen(oStudent.m_lpszGender)+1, oStudent.m_lpszGender);

    m_iScore = oStudent.m_iScore;
    cout << "调用了拷贝构造函数Student: " << m_lpszName << endl;
}

//Summary: 析构函数

```

```

//
//Parameters:
//      None
//Return: None
//Detail:
Student::~~Student()
{
    cout << "调用了析构函数Student: " << m_lpszName << endl;
    //释放姓名与性别字符串的内存空间
    delete []m_lpszName;
    delete []m_lpszGender;
}

//Summary: "="符号运算符重载函数
//
//Parameters:
//      oStudent    "="运算符的右值, Student类的对象
//Return: Student&    Student类的引用, 为左值赋值
//Detail:
Student &Student::operator=(const Student &oStudent)
{
    //如果语句的右值是本身, 则直接返回
    if (this == &oStudent)
    {
        return *this;
    }

    //重新分配内存空间, 并采用深拷贝方式复制学生姓名
    delete []m_lpszName;
    m_lpszName = new char[strlen(oStudent.m_lpszName)+1];
    if( m_lpszName==NULL )
    {
        cout << "内存分配失败。" << endl;
        exit(0);
    }
    strcpy_s(m_lpszName, strlen(oStudent.m_lpszName)+1, oStudent.m_lpszName);

    //重新分配内存空间, 并采用深拷贝方式复制学生性别
    delete []m_lpszGender;
    m_lpszGender = new char[strlen(oStudent.m_lpszGender)+1];
    if( m_lpszGender==NULL )
    {
        cout << "内存分配失败。" << endl;
        exit(0);
    }
}

```

```

    }
    strcpy_s(m_lpszGender, strlen(oStudent.m_lpszGender)+1, oStudent.m_lpszGender);

    m_iScore = oStudent.m_iScore;
    cout << "调用了重载'='运算符Student: " << m_lpszName << endl;
    return *this;
}

/*****
Filename: studentManagement.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2015-04-10
Description: StudentManagement类的定义

*****/
#include "Student_Management.h"
#include "Student.h"

//Summary: 构造函数
//
//Parameters:
//      iTotalStudentNum  栈能容纳的最大学生数目，默认值为P_LENGTH
//Return: None
//Detail:
StudentManagement::StudentManagement(int iTotalStudentNum)
{
    //对栈的空间进行初始化
    for (int i=0; i<iTotalStudentNum; i++)
    {
        m_pStudent[i] = NULL;
    }
    m_aLength = iTotalStudentNum;
    m_aCurrentTop = 0;
    cout << "调用了构造函数StudentManagement: " << endl;
}

//Summary: 拷贝构造函数
//
//Parameters:
//      oStudentManagement  待拷贝对象
//Return: None
//Detail:
StudentManagement::StudentManagement(const StudentManagement &oStudentManagement)

```

```

{
    //从待拷贝对象的栈中复制学生信息
    for (int i=0; i<oStudentManagement.m_aCurrentTop; i++)
    {
        m_pStudent[i] = new Student(oStudentManagement.m_pStudent[i]->m_lpszName,
                                     oStudentManagement.m_pStudent[i]->m_lpszGender,
                                     oStudentManagement.m_pStudent[i]->m_iScore);

        if (m_pStudent[i] == NULL)
        {
            cout << "内存分配失败!" << endl;
            exit(0);
        }
    }

    //将后续的学生信息初始化为空
    for (int i=oStudentManagement.m_aCurrentTop; i<oStudentManagement.m_aLength;
i++)
    {
        m_pStudent[i] = NULL;
    }

    //复制长度与栈顶位置
    m_aLength = oStudentManagement.m_aLength;
    m_aCurrentTop = oStudentManagement.m_aCurrentTop;
    cout << "调用了拷贝构造函数StudentManagement: " << endl;
}

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
//Detail:
StudentManagement::~StudentManagement(void)
{
    //释放所有栈中的学生信息对象
    for (int i=0; i<m_aCurrentTop; i++)
    {
        delete m_pStudent[i];
    }
    cout << "调用了析构函数StudentManagement。" << endl;
}

//Summary: 插入函数

```

```

//
//Parameters:
//      oStudent    待插入的学生信息对象
//Return: bool      插入是否成功
//Detail:
bool StudentManagement::Insert(const Student &oStudent)
{
    //检查栈容量
    if( m_aCurrentTop>=m_aLength )
    {
        cout << "对不起，已经超过了数组最大长度，不可再插入!" << endl;
        return false;
    }

    //创建新节点，复制学生信息，并插入栈中
    m_pStudent[m_aCurrentTop] = new Student(oStudent.m_lpszName,
oStudent.m_lpszGender, oStudent.m_iScore);
    if (m_pStudent[m_aCurrentTop] == NULL)
    {
        cout << "创建结点失败！" << endl;
        exit(0);
    }
    m_aCurrentTop++;
    cout << "插入了一个结点：" << oStudent.m_lpszName << endl;
    return true;
}

//Summary: 插入函数
//
//Parameters:
//      None
//Return: bool      删除是否成功
//Detail:
bool StudentManagement::Del(void)
{
    //检查栈是否为空
    if (!IsEmpty())
    {
        //若栈非空，删除栈顶的学生信息对象
        m_aCurrentTop--;
        cout << "删除的结点名字为：" << m_pStudent[m_aCurrentTop]->m_lpszName << endl;
        delete m_pStudent[m_aCurrentTop];
        return true;
    }
}

```

```

        else
        {
            cout << "结点数组为空。不能够执行删除操作。Is Empty\n";
            return false;
        }
    }

//Summary: 检查栈是否为空
//
//Parameters:
//      None
//Return: bool      栈是否为空的判断结果
//Detail:
bool StudentManagement::IsEmpty(void)
{
    //根据当前栈顶的位置判断
    if (m_aCurrentTop == 0)
    {
        return true;
    }
    else
    {
        return false;
    }
}

//Summary: ", "运算符重载函数
//
//Parameters:
//      oStudentManagement      ", "表达式右侧的参数
//Return: const StudentManagement &      ", "表达式的返回值，为const的原因是", "表达式返回
的必为右值
//Detail:
const StudentManagement & StudentManagement::operator, (const StudentManagement
&oStudentManagement)
{
    //直接返回右侧参数
    return oStudentManagement;
}

//Summary: "[]"运算符重载函数
//
//Parameters:
//      i      数组的下标

```

```

//Return: char * 当前下标的学生姓名
//Detail:
char *StudentManagement::operator[](int i)
{
    //判断下标是否越界
    if ((i < m_aCurrentTop)&&(m_aCurrentTop >=0))
    {
        //返回当前下标指向的学生姓名
        return m_pStudent[i]->m_lpszName;
    }
    else
    {
        return ("数组越界。");
    }
}

//Summary: "="运算符重载函数
//
//Parameters:
//      oStudentManagement    "="表达式的右侧对象
//Return: StudentManagement & 赋值结果
//Detail:
StudentManagement & StudentManagement::operator=(const StudentManagement
&oStudentManagement)
{
    //判断，如果是自身向自身赋值，则直接返回
    if (this == &oStudentManagement)
    {
        return *this;
    }
    //判断两栈长度是否相等
    if (m_aLength != oStudentManagement.m_aLength)
    {
        cout << "数组长度不等，不能够复制!" << endl;
        return *this;
    }

    //清空当前栈的内容
    for (int i=0; i<m_aCurrentTop; i++)
    {
        delete m_pStudent[i];
        m_pStudent[i] = NULL;
    }
}

```

```

//对每个学生信息对象进行深拷贝复制
for (int i=0; i<oStudentManagement.m_aCurrentTop; i++)
{
    m_pStudent[i] = new Student(oStudentManagement.m_pStudent[i]->m_lpszName,
                                oStudentManagement.m_pStudent[i]->m_lpszGender,
                                oStudentManagement.m_pStudent[i]->m_iScore);
    if (m_pStudent[i] == NULL)
    {
        cout << "内存分配失败!" << endl;
        exit(0);
    }
}

//将栈未使用的部分置为NULL
for (int i=oStudentManagement.m_aCurrentTop; i<oStudentManagement.m_aLength;
i++)
{
    m_pStudent[i] = NULL;
}

//复制长度与当前栈顶值
m_aLength = oStudentManagement.m_aLength;
m_aCurrentTop = oStudentManagement.m_aCurrentTop;
cout << "调用重载运算符" << endl;
return *this;
}

//Summary: "new"运算符重载函数
//
//Parameters:
//      size    内存空间大小
//Return: void * 分配到的内存地址
//Detail:
void *StudentManagement::operator new(size_t size)
{
    cout << "自定义的StudentManagement new\n";
    return malloc(size);
}

//Summary: "delete"运算符重载函数
//
//Parameters:
//      p      待释放的内存地址
//Return: None

```

```

//Detail:
void StudentManagement::operator delete(void *p)
{
    cout << "自定义的StudentManagement delete\n";
    free(p);
}

//Summary: "<<"运算符重载函数
//
//Parameters:
//      scout      输出流
//      obj        待输出的对象
//Return: ostream  输出流
//Detail:
ostream &operator << (ostream & scout, StudentManagement obj)
{
    //输出栈顶学生的姓名，性别，成绩
    scout << obj.m_pStudent[obj.m_aCurrentTop-1]->m_lpszName << ",";
    scout << obj.m_pStudent[obj.m_aCurrentTop-1]->m_lpszGender << ",";
    scout << obj.m_pStudent[obj.m_aCurrentTop-1]->m_iScore << ",";
    //输出栈的长度与当前栈顶位置
    scout << obj.m_aLength << ",";
    scout << obj.m_aCurrentTop << endl;
    return scout;
}

//Summary: ">>"运算符重载函数
//
//Parameters:
//      scin       输入流
//      obj        待输入的对象
//Return: istream  输入流
//Detail:
istream &operator >> (istream &scin, StudentManagement &obj)
{
    int iTemp = 0;
    //为输入的姓名，性别分配内存空间
    char *lpszName = new char[100];
    char *lpszGender = new char[100];

    //输入待操作下标并判断是否越界
    cout << "请输入操作下标: ";
    scin >> iTemp;
    if( iTemp>=obj.m_aCurrentTop )

```

```

{
    cout << "你输入的节点下标大于当前节点值：" << obj.m_aCurrentTop << endl; //
    cout << "对不起，你的操作是错误的！" << endl;
    exit(0);
}

//输入学生成绩
cout << "请输入学生成绩：" ;
scin >> obj.m_pStudent[iTemp]->m_iScore;

//输入学生姓名，并完成替换
cout << "原学生姓名为：" << obj.m_pStudent[iTemp]->m_lpszName << endl;
cout << "请输入新学生姓名：" << endl;
scin.get();
scin.getline(lpszName, 20);
delete []obj.m_pStudent[iTemp]->m_lpszName;
obj.m_pStudent[iTemp]->m_lpszName = lpszName;
cout << "现学生姓名为：" << obj.m_pStudent[iTemp]->m_lpszName << endl;

//输入学生性别，并完成替换
cout << "原学生性别为：" << obj.m_pStudent[iTemp]->m_lpszGender << endl;
cout << "请输入新学生性别：" << endl;
//scin.get();
scin.getline(lpszGender, 20);
delete []obj.m_pStudent[iTemp]->m_lpszGender;
obj.m_pStudent[iTemp]->m_lpszGender = lpszGender;
cout << "现学生性别为：" << obj.m_pStudent[iTemp]->m_lpszGender << endl;
return scin;
}

```

2.4 综合程序应用——犀利人事管理系统

在前面一章的综合程序应用的基础上，对“犀利人事管理系统”进行功能扩展和修改。

要求：

- (1) 将公司员工增加到 20 人，并采用对象数组表示。
- (2) 对>> 和<<进行重载。重载后使用“>>”可以从键盘输入一个员工对象的所有信息。

```

*****
Filename: main.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: 程序主函数文件

```

```

*****/
#include <iostream>
#include <fstream>
#include "employee.h"
using namespace std;

//Summary: 主函数
//
//Parameters:
//      None
//Return: int 一般为
int main()
{
    //使用对象数组存储人的雇员信息
    employee empArray[20];

    //使用文件进行个人信息的输入、输出操作
    ifstream inputFile("emp_info.txt", ios::in);
    ofstream outputFile("emp_output.txt", ios::out);

    int count;
    //重载流输入操作符完成个人信息输入
    for(count = 0; count < 20; count++)
        inputFile>>empArray[count];

    //重载流输出操作符完成个人信息输出
    for(count = 0; count < 20; count++)
        outputFile<<empArray[count];

    //关闭文件
    inputFile.close();
    outputFile.close();

    return 0;
}

/*****
Filename: employee.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: employee类的声明

```

```

*****/
#ifndef EMPLOYEE_H
#define EMPLOYEE_H

//employee类，对公司的雇员信息进行封装
class employee
{
private:
    //三个私有数据成员，分别代表雇员的编号(individualEmpNo)、等级(grade)与月薪(accumPay)
    int individualEmpNo;
    int grade;
    int accumPay;

    //这个静态私有成员用于在新加入雇员时使编号自动增加
    static int currentEmpNo;
public:
    //无参构造函数与有参构造函数
    employee();
    employee(int inputGrade, int inputPay);

    //析构函数
    ~employee();

    //访问数据成员的接口函数，分别是
    //设置等级: setGrade(int)
    //设置月薪: setAccumPay(int)
    //获取编号: int getEmpNo()
    //获取等级: int getGrade()
    //获取月薪: int getAccumPay()
    //注: 雇员编号是构造时自动生成的，为避免编号冲突，操作者无法设置编号
    void setGrade(int inputGrade) {grade = inputGrade;};
    void setAccumPay(int inputPay) {accumPay = inputPay;};
    int getEmpNo() {return individualEmpNo;};
    int getGrade() {return grade;};
    int getAccumPay() {return accumPay;};

    //该函数的功能是打印雇员的基本信息
    void printInfo();

    //流输入运算符重载，从流向对象输入等级与月薪
    friend std::istream& operator >>(std::istream&, employee&);

    //流输出运算符重载，向流输出对象的基本信息

```

```

        friend std::ostream& operator <<(std::ostream&, employee&);
};

#endif

*****
Filename: employee.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: employee类的定义

*****/
#include <iostream>
#include "employee.h"
using namespace std;

//设置静态变量的初始值
int employee::currentEmpNo = 2014001;

//Summary: 输出雇员的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 该函数首先输出一条提示信息，然后按照
//      编号、等级与月薪的顺序输出雇员的基本信息
void employee::printInfo()
{
    cout<<"输出个人信息: "<<endl;
    cout<<"编号: "<<individualEmpNo<<endl;
    cout<<"等级: "<<grade<<endl;
    cout<<"月薪: "<<accumPay<<endl<<endl;
}

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
//Detail: 无参构造函数首先完成新雇员的自动编号功能
//      再将雇员的等级设为，月薪设为
employee::employee()
{

```

```

        individualEmpNo = currentEmpNo;
        currentEmpNo++;
        grade = 1;
        accumPay = 3000;
    }

//Summary: 有参构造函数
//
//Parameters:
//    inputGrade: 输入的雇员等级
//    inputPay:    输入的雇员月薪
//Return: None
//Detail: 有参构造函数首先完成新雇员的自动编号功能
//    之后根据输入的参数设置雇员等级与月薪
//注:    未添加合法性判断, 请勿输入非正数
employee::employee(int inputGrade, int inputPay)
{
    individualEmpNo = currentEmpNo;
    currentEmpNo++;
    grade = inputGrade;
    accumPay = inputPay;
}

//Summary: 析构函数
//
//Parameters:
//    None
//Return: None
//Detail: 释放对象, 并输出提示信息
employee::~employee()
{
    cout<<"欢迎使用, 再见!"<<endl;
}

//Summary: 流输入操作符重载函数
//
//Parameters:
//    input: 输入流
//    e:     目标雇员对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示, 之后从流中输入目标对象的等级与月薪
//注:    未添加合法性判断, 请勿输入非正数
istream& operator >> (istream& input, employee& e)

```

```

{
    cout<<"输入编号为"<<e.individualEmpNo<<" 的雇员的等级、月薪："<<endl;
    input>>e.grade;
    input>>e.accumPay;
    return input;
}

//Summary: 流输出操作符重载函数
//
//Parameters:
//    output: 输出流
//    e:      目标雇员对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, employee &e)
{
    output<<"输出个人信息："<<endl;
    output<<"编号："<<e.individualEmpNo<<endl;
    output<<"等级："<<e.grade<<endl;
    output<<"月薪："<<e.accumPay<<endl<<endl;
    return output;
}

```

本章小结

在本章中，我们学习了 C++ 语言的第 2 大重要特点，运算符和函数的重载性。在 C++ 中，运算符重载是很重要的、很有实用意义的。它使类的设计更加丰富多彩，扩大了类的功能和使用范围，使程序易于理解，易于对对象进行操作。它体现了 C++ 代码的可扩展性和重用性的思想。有了运算符重载，在声明了类之后，人们就可以像使用标准类型一样来使用自己声明的类。类的声明往往是一劳永逸的，有了好的类，用户在程序中就不必定义许多成员函数去完成某些运算和输入输出的功能，使主函数更加简单易读。好的运算符重载能体现面向对象程序设计思想。

另外，本章还学习一个核心概念，类型转换函数。采用类型转换函数可以简化代码的编写。利用隐式自动类型转换，可以将原本多个运算符重载函数合并为一个。例如：

```

friend Vector operator + (Vector vec1, Vector vec2);
friend Vector operator + (Vector vec1, double vec2);
friend Vector operator + (double vec1, Vector vec2);
...

```

可以采用下列 2 条语句来替代。

```
Vector (double d);
```

```
friend Vector operator + (Vector vec1, Vector vec2);
```

另一方面，使用隐式的自动类型转换给程序带来隐患。建议编程时，尽量多使用显式类型转换。例如 `vec3=vec1+5.5`（隐式转换），写成 `vec3=vec1+Vector(5.5)`（显式转换）。

课后习题

1. 一个类中的友元函数是否可以申明在类中的 `private`？类中申明的友元函数是否有 `this` 指针？
2. 分析复制构造函数与类型转换构造函数的区别与联系？总结一下到目前为止，我们学习了哪些类型的构造函数？各有什么特点？
3. 在运算符重载时，如何使得重载后运算符在实际运算时，能符合交换律？总结一下能用几种方案来实现。
4. 设计一个用于人事管理的 `People` (人员) 类，具有的属性如下：姓名 `char name[11]`、编号 `char number[7]`、性别 `char sex[3]`、生日 `birthday`、身份证号 `char id[16]`。其中“出生日期”声明为一个“日期”类内嵌子对象。用成员函数实现对人员信息的录入和显示。要求包括：构造函数和析构函数、拷贝构造函数、内联成员函数、运算符重载等。在测试程序中声明。
5. 定义一个 `Teacher` 类和一个 `Student` 类，两者有一部分的数据成员是相同的，例如：`num`, `name` 和 `sex`。编写程序 将一个 `Teacher` 类对象转为 `Student` 类对象，只需将以上 3 个相同的数据成员移植过去。
6. 编写一个程序，用成员函数重载运算符“+”和“-”将 2 个二维数组相加和相减，要求第一个二维数据的值由构造函数来设定，另一个二维数据的值由键盘输入。
7. 修改第 5 题程序，用友元函数重载运算符“+”和“-”将 2 个二维数组相加和相减，要求 2 个二维数据的值都由键盘输入。
8. 分析本章中 2.4 节中“人事管理系统”的程序代码，划出程序设计基本框架，分析该程序使用了本章学习到的哪些知识点。